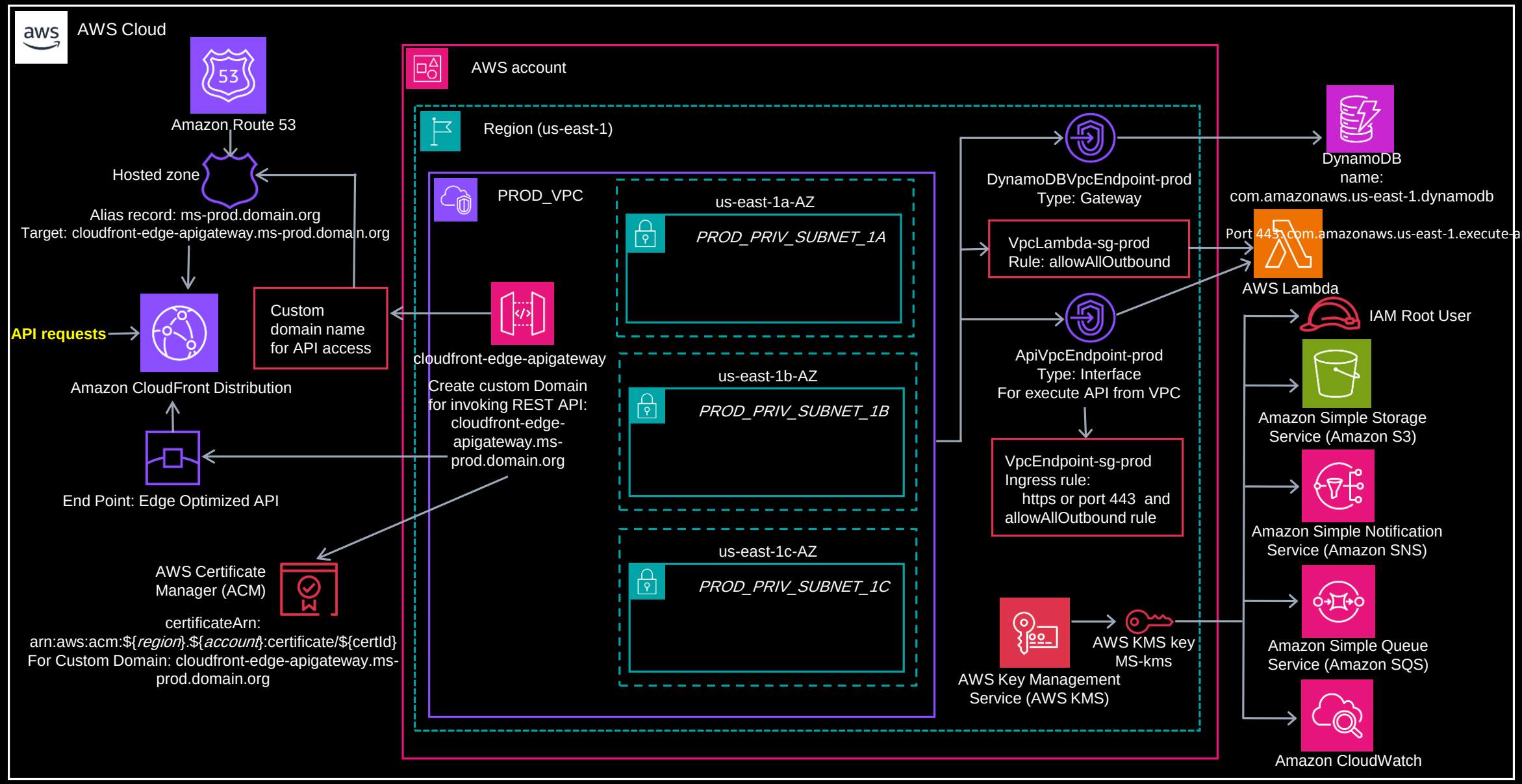
Max Montaseri

# AWS Infrastructure as Code

# CDK Infrastructure Pipeline

aws

When you create a custom domain name for an edge-optimized API, API Gateway sets up a CloudFront distribution. But you must set up a DNS record to map the custom domain name to the CloudFront distribution domain name. This mapping is for API requests that are bound for the custom domain name to be routed to API Gateway through the mapped CloudFront distribution. You must also provide a certificate for the custom domain name.



**types.ts**

```
import { StackProps, StageProps } from 'aws-cdk-lib'

InfrastructurePipelineStackProps
  -vpcId: string
  -waves: Array<InfrastructurePipelineStageProps>
                -approval?: boolean
                -name: string
                -environments: Array<InfrastructureStackProps>
                                        -vpcId: string
                                        -stackEnv: string
                                        -domain: InfrastructureDomainProps
                                                  -domainName: string
                                                  -certId: string

                                        -subnets: Array<InfrastructureSubnetProps>
                                                            -subnet: string
                                                            -zone: string
```

aws

## config.ts

```
const MS_NAME='order'
const ENV_PROD = { account: '3??????????6', region: 'us-east-1' }
const PROD_ROOT_DOMAIN = `${MS_NAME}-prod.domain.org`
// AWS Certificate Manager (ACM): already should be created in account (free tire: first 30 days)
const PROD_ACM_CERT_ID = '8d9506d5-????-4625-????-af2da279????'
//'vpc-prod' // alreday should be created in account
const PROD_VPCID = 'vpc-????82e21bec26d05'
const PROD_PRIV_SUBNET_1A = 'prod-subnet-1a'
const PROD_PRIV_SUBNET_1B = 'prod-subnet-1b'
const PROD_PRIV_SUBNET_1C = 'prod-subnet-1c'

const ENV_NONPROD = { account: '????7645????', region: 'us-east-1' }
const NONPROD_ROOT_DOMAIN = `${MS_NAME}-nonprod.domain.org`
const NONPROD_ACM_CERT_ID = '8d9506d5-????-4625-????- ????a2792e44‘
//'vpc-nonprod' // already should be created in account
const NONPROD_VPCID = 'vpc-????980????1908‘
const NONPROD_PRIV_SUBNET_1A = 'nonprod-subnet-1a'
const NONPROD_PRIV_SUBNET_1B = 'nonprod-subnet-1b'
const NONPROD_PRIV_SUBNET_1C = 'nonprod-subnet-1c'

export const ZSCALER_IPSET_1 = '10.132.8.0/21'
export const ZSCALER_IPSET_2 = '10.132.0.0/16'

export const PIPELINE_NAME = `${MS_NAME.toUpperCase}-InfrastructurePipeline`
export const ASSET_NAME = `${MS_NAME}-infrastructure`
export const REFERENCE_CDN_PREFIX = 'cloudfront-edge-apigateway'
```

**config.ts**

```ts
export const getTags = (env: string) => ({
  asset_id: ASSET_NAME,
  system_id: `${MS_NAME}`,
  accessibility: 'private',
  data_classification: 'proprietary',
  env_intent: env,
  engineer: 'devops@company.org',
})
export const PIPELINE_CONFIG = {
  env: ENV_NONPROD,
  tags: getTags('nonprod'),
  vpcId: NONPROD_VPCID,
  waves: [
    {
      environments: [
        {
          stackEnv: 'nonprod',
          subnets: [
            { subnet: NONPROD_PRIV_SUBNET_1A, zone: 'us-east-1a' },
            { subnet: NONPROD_PRIV_SUBNET_1B, zone: 'us-east-1b' },
            { subnet: NONPROD_PRIV_SUBNET_1C, zone: 'us-east-1c' },
          ],
          domain: {
            certId: NONPROD_ACM_CERT_ID,
            domainName: NONPROD_ROOT_DOMAIN,
          },
          vpcId: NONPROD_VPCID,
        },
      ],
      name: 'nonprod-stage',
      env: ENV_NONPROD,
      tags: getTags('nonprod'),
    },
    {
      environments: [
        {
          stackEnv: 'prod',
          subnets: [
            { subnet: PROD_PRIV_SUBNET_1A, zone: 'us-east-1a' },
            { subnet: PROD_PRIV_SUBNET_1B, zone: 'us-east-1b' },
            { subnet: PROD_PRIV_SUBNET_1C, zone: 'us-east-1c' },
          ],
          domain: {
            certId: PROD_ACM_CERT_ID,
            domainName: PROD_ROOT_DOMAIN,
          },
          vpcId: PROD_VPCID,
        },
      ],
      approval: true,
      name: 'prod-stage',
      env: ENV_PROD,
      tags: getTags('prod'),
    },
  ],
}
```

aws

**InfrastructurePipelineStack (CodeCommit, CodeBuild and CodePipeline)** will be executed whenever `PIPELINE_CONFIG` or `config.ts` configurations are changed in CodeCommit repository.

Then reads the pipeline's waves (nonprod and prod) and their environments (StackEnv, VPC, Subnets, Domain) from `PIPELINE_CONFIG.waves`

Finally invokes the InfrastructureStack for provisioning each environment infrastructure

```
InfrastructurePipelineStack
  const { waves, vpcId } = props || {}
  const vpc = Vpc.fromLookup(this, 'vpc',  {vpcId : vpcId })
  const pipeline = new CodePipeline(this, 'Pipeline', {
      selfMutation: false,
      pipelineName: PIPELINE_NAME,
      crossAccountKeys: true,
      // Synth: will take our codes then generate AWS CloudFormation templates.
      // we can see the generated templates. this happens always we deploy!
      synth: new CodeBuildStep('Synth', {
        projectName: ASSET_NAME,
        input: CodePipelineSource.codeCommit(Repository.fromRepositoryName(this, 'repo', ASSET_NAME), 'master'),
        commands: ['npm ci', 'npm run build', 'npx cdk synth'],
        vpc,
      }),
    })

  waves.forEach((w) => {
    const { environments, approval, name = '', ...common } = w
    const wave = pipeline.addWave(name, {
      pre: approval ? [new ManualApprovalStep(`promoteto${name}`)] : undefined,
    })

    environments.forEach((e) => {
      wave.addStage(new InfrastructurePipelineStage(this, `${ASSET_NAME}-${e.stackEnv}`, { ...common, ...e }))
    })
  })
```

aws

**InfrastructurePipelineStage** invoked by **Infrustructure**PipelineStack for each pipeline stage. then invokes InfrastructureStack for provisioning infrastructures for each stage's environments regarding each environment (StackEnv, VPC, Subnets and Domain)

```
new InfrastructureStack(this, 'Infrastructure-stack', props)
```

**InfrastructureStack** provisions AWS services as cloud infrastructure for each stage (prod, nonprod) and it's environments (Dev, SysTest, UAT, DR, Sandbox)

1- *createGatewayVPCEndpoint*(vpc)
creates two security groups for both prod/nonprod VPCs with allowAllOutbound as well as InbounRules

*vpcEndpoint-sg-nonprod & vpcEndpoint-sg-prod => HTTPS =>* `addIngressRule(Peer.ipv4(vpc.vpcCidrBlock), Port.tcp(443))`
*vpcEndpoint-sg-nonprod => HTTPS =>* `addIngressRule(Peer.ipv4(ZSCALER_IPSET_1), Port.tcp(443))`
`addIngressRule(Peer.ipv4(ZSCALER_IPSET_2), Port.tcp(443))`

create an interface VPC endpoint for the API Gateway component service for API execution, known *as* execute-api, in both prod/nonprod VPCs and their subnets for *HTTPS =>* `Port.tcp(443))` with name: 'com.amazonaws.us-east-1.execute-api'
ApiVpcEndpoint-nonprod & ApiVpcEndpoint-prod

2- *createSecurityGroup*(vpc, `vpcLambda-sg-${stackEnv}`)
creates two security groups for AWS Lambda for both prod/nonprod VPCs with allowAllOutbound only (not inbound rules!)
*vpcLambda-sg-nonprod & vpcLambda-sg-prod*

3-*createDynamoDbVPCEndpoint*(vpc)
create two *GatewayVpcEndpoint* for Amazon DynamoDB, in both prod/nonprod VPCs
DynamoDBVpcEndpoint-nonprod & DynamoDBVpcEndpoint-prod

*Gateway VPC endpoints allow you to connect to Amazon Simple Storage Service (Amazon S3) and Amazon DynamoDB privately from your VPC.*

aws

## InfrastructureStack

```
4-createKMSKey()

   Create a KMS Key with key rotation for only Decrypting action in Order-microservice integration
   using related SNS, SQS, S3 and CloudWatch services (principals), also all actions allowed
   in KMS for AWS account root user

5-createAPIGatewayCustomDomain(REFERENCE_CDN_PREFIX)

   Create prod/nonprod custom domain name (certificateArn) with EDGE type for API Gateway:
      domainName: apigateway.domain.nonprod
      domainName: apigateway.domain.prod
      certificateArn: arn:aws:acm:region:account:certificate/certId

   Then add prod/nonprod alias records (apigateway.domain.nonprod & apigateway.domain.prod)
   to Route53 prod/nonprod hosted zones (domain.nonprod & domain.prod)

   These prod/nonprod alias records target APIGateway regarding created custom domains (domainName
   & certificateArn)

6-dmsCloudwatchLogRole = new iam.Role(this, 'dms-cloudwatch-logs-role', {
  // Database Migration Service (DMS) Role
  // TODO: Once migration is complete, should be removed
  add a new DMS (dms.amazonaws.com) service role (dms-cloudwatch-logs-role) to IAM
  for sending database migration (only once) logs to CloudWatch
```